TP 4 - slides annexes

Héritage, interfaces, classes abstraites et polymorphisme

Olivier Goudet

Université d'Angers

28/02/2022

Section 1

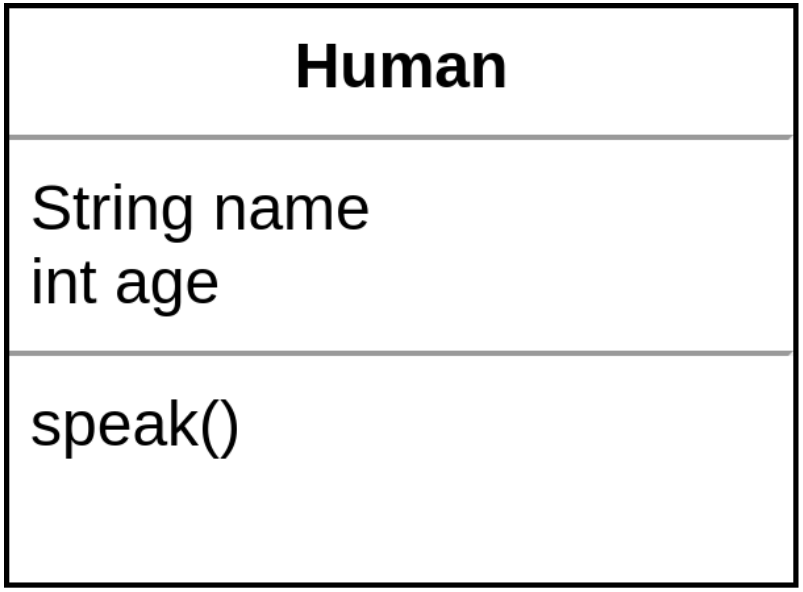
Les classes en Java

Définir une classe en Java

```
public class Human {

        //Attributs
        private String name;
        private int age;

        //Constructeur
        public Human(String name, int age){
                this.name = name;
                this.age = age;
        }

        //Methode
        public void speak(){
                System.out.println("Hello, my name is " + name);
                System.out.println("I am " + age + " years old");
        }
}
```

Diagramme de classes UML simplifié

On va utiliser une version simplifiée des diagrammes de classes UML dans cette UE, telle qu'utilisée dans le livre de référence *Tête la première - Design pattern* (disponible sur Moodle).



Nom de la classe

Attributs

Méthodes

Création d'objets en Java

```
public class World {

        public static void main(String args[]){

                Human human1 = new Human("Jack", 25);
                Human human2 = new Human("John", 32);

                human1.speak();
                human2.speak();

        }
}
```

Sorties console

Hello, my name is Jack
I am 25 years old
Hello, my name is John
I am 32 years old

Section 2

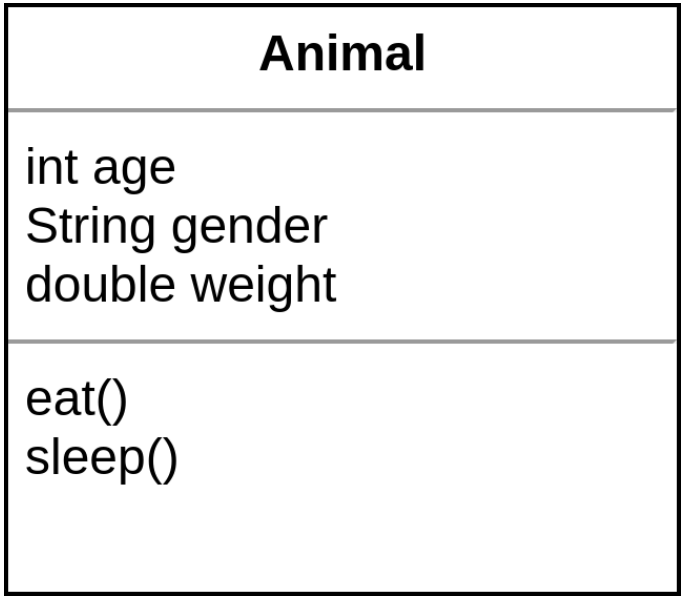
L'héritage en Java

L'héritage en Java

- **Définition :** le terme héritage désigne le principe selon lequel une classe peut hériter des caractéristiques (attributs et méthodes) d'autres classes.
- Soient deux classes A et B. Relation d'héritage : Classe B "EST UN" Classe A.
  - A est la super-classe ou classe mère de B
  - B est la sous-classe ou classe fille de A

Pourquoi utiliser l'héritage ?

- Une classe décrit des services : fonctionnalités et données. Par exemple ci-dessous, un animal est caractérisé par un age, un genre et un poids. Il peut manger et dormir.
- Pour implémenter différents types d'animaux, deux problèmes peuvent apparaître :
  - la méthode *eat()* peut différer suivant les animaux.
  - Il manque des services spécifiques : *fly()*, *swim()*.



Mauvaises solutions

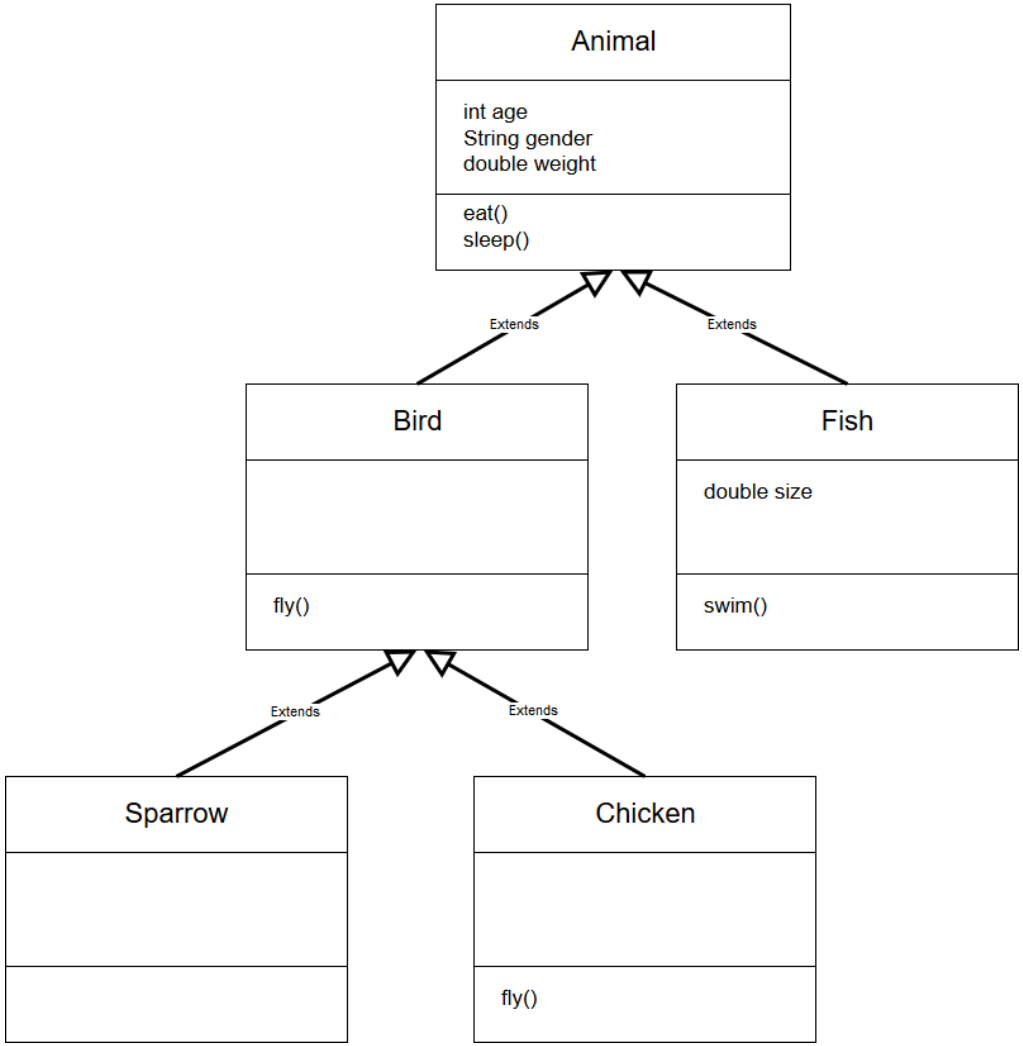
- Faire autant de classes que d'espèces animales différentes.
  - beaucoup de services vont être très similaires entre les classes.
  - le copier-coller est toujours dû à une erreur de conception.

  $\rightarrow$ **coût énorme pour maintenir le code**
- Modifier la classe animal pour représenter tous les animaux.
  - ajouter des méthodes dans la classe Animal pour toutes les actions des animaux : *fly()*, *swim()*
  - Ajouter un attribut qui indique la catégorie d'animal.
  - Implémenter des fonctions en testant l'attribut qui indique la catégorie et réaliser une action spécifique en fonction de cette catégorie.

  $\rightarrow$ **coût énorme pour étendre le code**

La solution avec l'héritage

- Utiliser une hiérarchie de classe et faire de l'héritage.
- Une classe hérite de l'ensemble des services de ses ascendants.



Héritage en Java

Trois règles en Java :

1. Une classe hérite toujours d'une seule classe.
2. Par défaut une classe hérite de la classe *Object* et donc de toutes les méthodes de cette classe *Object*.
   - Exemple : la méthode *toString()* de la classe *Object*. Elle renvoie le nom de la classe de l'objet concerné suivi de l'adresse de cet objet.
   - On peut redéfinir cette méthode *toString()* dans les classes filles, de façon à renvoyer une chaîne de caractères qui décrit mieux l'objet en question.
3. Les cycles sont interdits dans les relations d'héritage.

Héritage - exemple en Java

```
public class Animal {

        int age;
        String gender;
        double weight;

        public Animal(int age, String gender, double weight){
                this.age = age;
                this.gender = gender;
                this.weight = weight;
        }

        public void eat(){
                System.out.println("Eating...");
        }

        public void sleep(){
                System.out.println("Sleeping...");
        }

}
```

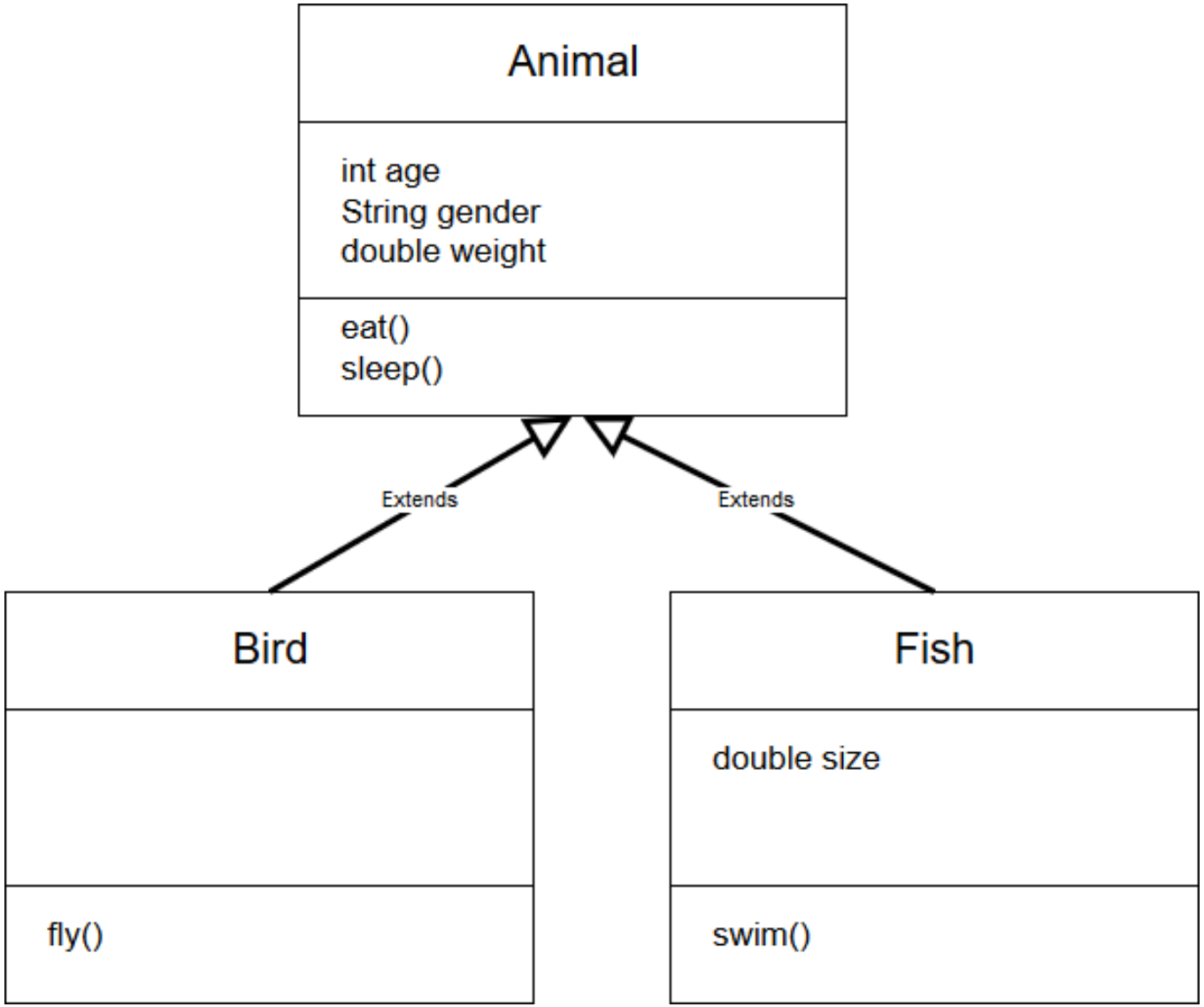
La classe *Bird* étend la classe *Animal*

```
public class Bird extends Animal{

        public Bird(int age, String gender, double weight) {
                super(age, gender, weight);
        }

        public void fly(){
                System.out.println("Flying...");
        }
}
```

La classe *Fish* étend la classe *Animal*

```
public class Fish extends Animal{

        double size;

        public Fish(int age, String gender, double weight, double size) {
                super(age, gender, weight);
                this.size = size;
        }

        public void swim(){
                System.out.println("Swimming...");
        }

}
```

Diagramme de classes UML simplifié avec une relation d'héritage



Hériter d'une classe déjà héritée

```
public class Sparrow extends Bird{

        public Sparrow(int age, String gender, double weight) {
                super(age, gender, weight);
        }

}
```

Test

```
public class Zoo {

        public static void main(String args[]){

                Sparrow sparrow = new Sparrow(13, "M", 500);
                sparrow.fly();

        }
}
```

Console

Flying...

Rappel : héritage et encapsulation

- Quatre niveaux d'accessibilité des différents attributs des classes :
  1. Un attribut ou une méthode déclarée **private** n'est accessible que depuis l'intérieur de la même classe.
  2. Un attribut ou une méthode déclarée **protected** est accessible uniquement aux classes d'un package et à ses sous-classes même si elles sont définies dans un package différent.
  3. Une classe, un attribut ou une méthode déclarée **public** est visible par toutes les classes et les méthodes.
  4. Si aucun modificateur n'est indiqué, l'élément n'est accessible que depuis les classes faisant partie du même package.

Signature, redéfinition et surcharge de méthodes

- La **signature d'une méthode** est composée de son nom et des types de ses arguments.
- **Redéfinition d'une méthode** : possibilité de définir le comportement d'une méthode selon le type d'objet l'invoquant, i.e., de donner une nouvelle implémentation à une méthode héritée sans changer sa signature.
- **Surcharge d'une méthode** : possibilité de définir des comportements différents pour la même méthode selon les arguments passés en paramètres. On change la signature de la méthode.

Redéfinition d'une méthode de la classe mère

```
public class Chicken extends Bird{

        public Chicken(int age, String gender, double weight) {
                super(age, gender, weight);
        }

        //Redefinition de la methode fly de la classe Bird
        public void fly(){
                System.out.println("not able to fly...");
        }
}
```

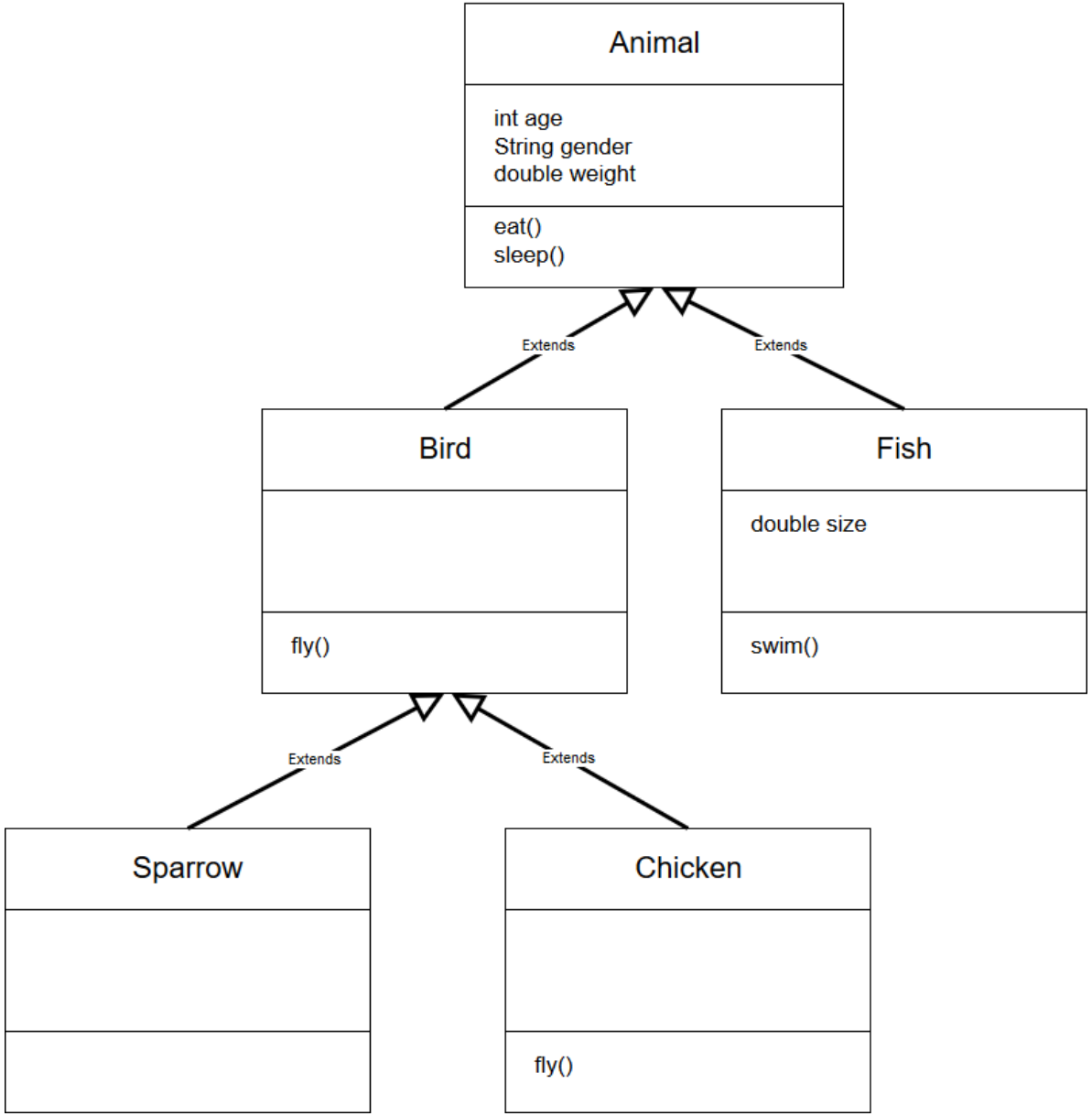
Exemple : redéfinition d'une méthode de la classe mère

```
public class Zoo {

        public static void main(String args[]){

                Chicken chicken = new Chicken(13, "M", 500);
                chicken.fly();

        }
}
```

Console

not able to fly...

Diagramme de classes UML simplifié



Section 3

Les interfaces en Java

Définir une interface

- Le code qu'on a écrit fonctionne, mais n'est pas très propre, car on doit redéfinir une méthode *fly()* pour la classe Chicken, alors qu'un poulet ne vole pas (ou pas bien)...
- Une autre façon de faire est d'enlever la méthode fly() de la classe *Bird* car tous les oiseaux ne volent pas.
- On définit ensuite une interface *Flyable* qui précise le comportement des animaux qui volent.
- C'est un "contrat" que les oiseaux volant doivent remplir.

Avantage des interfaces

- Permet d'avoir une vision claire de la structure du code.
- On sait clairement quels sont les animaux qui volent. Ce sont ceux qui implémentent *Flyable*.
- Permet de définir un super-type *Flyable*.
- Par exemple, on peut définir un objet avion qui implémente *Flyable*. Il vole aussi mais n'est pas nécessairement un animal.
- On pourra utiliser des fonctionnalités communes de tous les objets qui volent grâce au polymorphisme.

Programmer une interface

```
public class Bird extends Animal{

        public Bird(int age, String gender, double weight) {
                super(age, gender, weight);
        }

        //Suppression de la methode fly de la classe Bird

        //public void fly(){
        // System.out.println("Flying...");
        //}

}
```

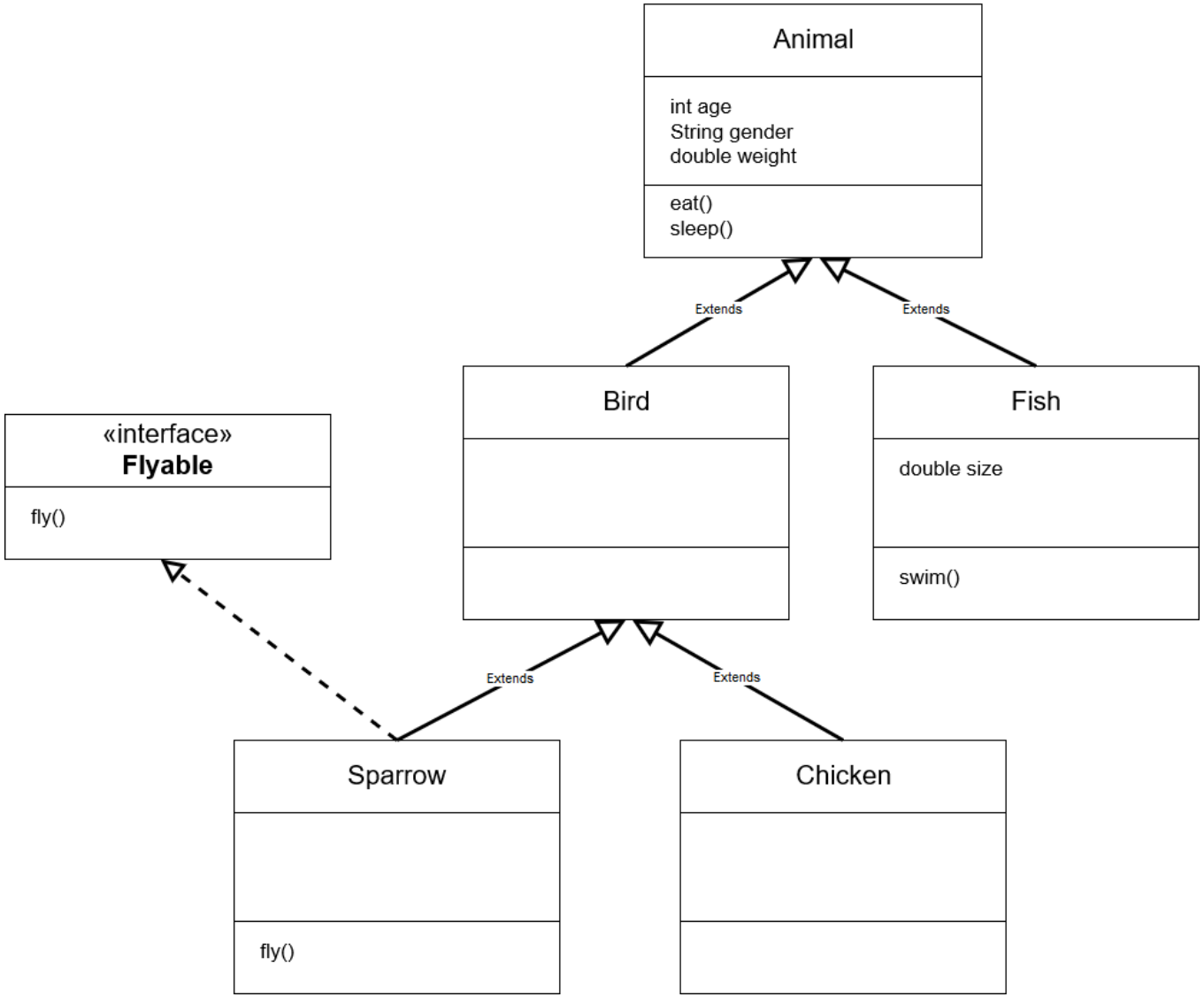
Programmer une interface

```
public interface Flyable {
        public void fly();
}
```

Programmer une interface

```
public class Sparrow extends Bird implements Flyable{

        public Sparrow(int age, String gender, double weight) {
                super(age, gender, weight);
        }

        public void fly() {
                System.out.println("Sparrow flying...");
        }

}
```

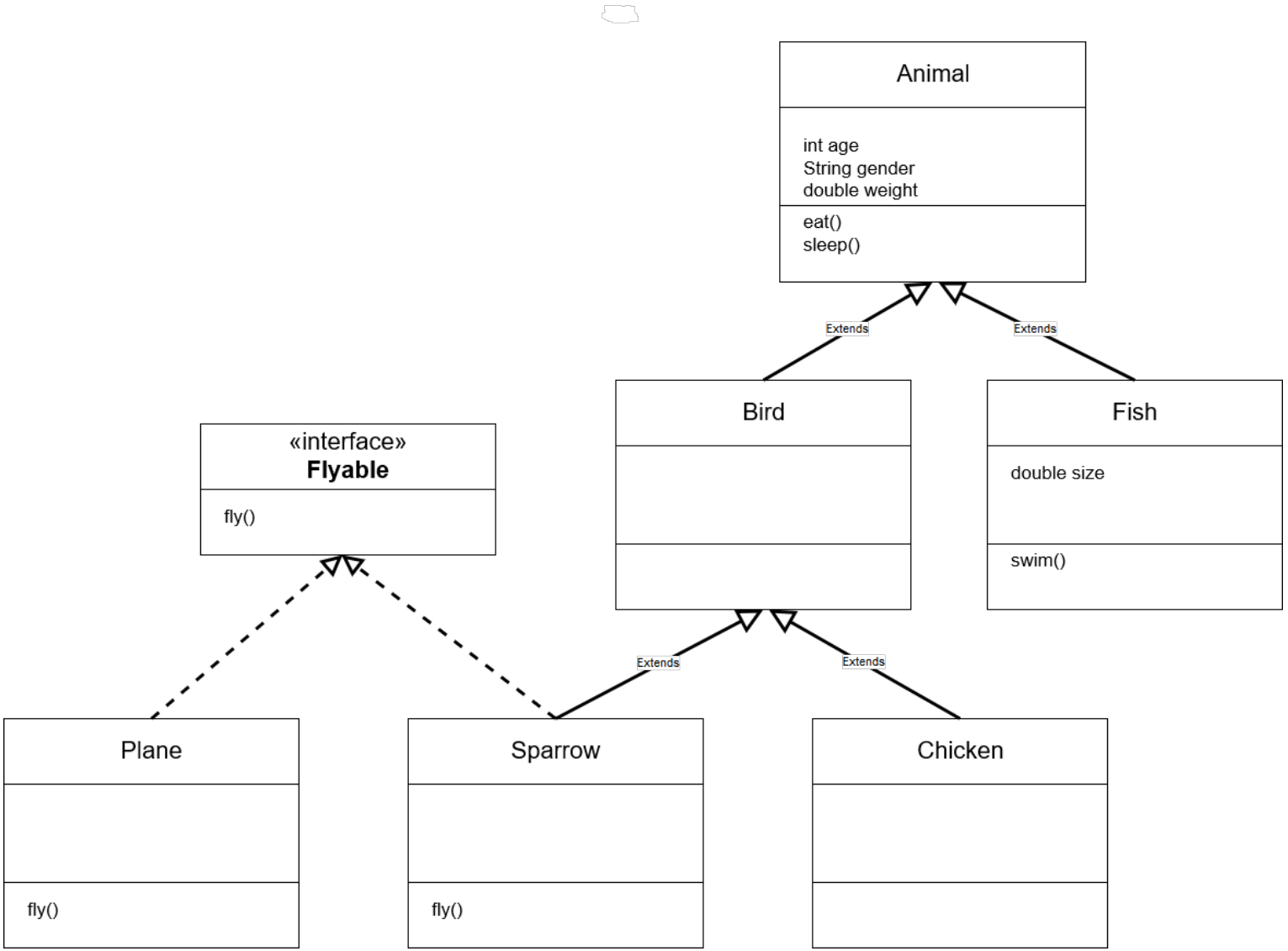
Diagramme de classes UML simplifié



Un autre objet qui vole mais n'est pas un oiseau...

```
public class Plane implements Flyable{
        public void fly() {
                System.out.println("Plane flying....");
        }
}
```

Diagramme UML



Petit Quizz. Est-ce que ces codes sont corrects ?

```
public class Zoo {
        public static void main(String args[]){

                Animal animal = new Animal(13, "M", 500);
                animal.eat();
                animal.fly();

                Bird bird = new Bird(5, "F", 100);
                bird.eat();
                bird.fly();

                Fish fish = new Animal(8, "M", 200);
                Fish fish2 = (Fish) new Animal(8, "M", 200);
                fish2.swim();
                fish2.fly();

                Animal fish3 = new Fish(8, "M", 200, 700);
                fish3.eat();
                fish3.swim();
                ((Fish) fish3).swim();
        }
}
```

Petit Quizz. Réponses

```
public class Zoo {
        public static void main(String args[]){

                Animal animal = new Animal(13, "M", 500); //OK
                animal.eat(); //OK
                animal.fly(); //Error

                Bird bird = new Bird(5, "F", 100); //OK
                bird.eat(); //OK
                bird.fly(); //Error

                Fish fish = new Animal(8, "M", 200); //Error
                Fish fish2 = (Fish) new Animal(8, "M", 200); //OK
                fish2.swim(); //OK
                fish2.fly(); //Error

                Animal fish3 = new Fish(8, "M", 200, 700); //OK
                fish3.eat(); //OK
                fish3.swim(); //Error
                ((Fish) fish3).swim();//OK
        }
}
```

Polymorphisme

- Le polymorphisme veut dire que le même service, aussi appelé opération ou méthode, peut avoir un comportement différent selon les situations.
- Si la classe B hérite de la classe A.
  - Classe B "EST-UN" Classe A.
  - Toute méthode *m* de A peut-être invoquée sur une instance de la classe B.
- Le polymorphisme consiste à exploiter cela en fournissant un B dans les expressions "qui attendent" un A.

Exemple : on peut regrouper les objets qui volent dans un tableau d'objets volants (1/2)

```
public class Test {

        public static void main(String args[]){

                Flyable sparrow = new Sparrow(13, "M", 500);
                Flyable plane = new Plane();

                Flyable flyingObjects[] = new Flyable[2];

                flyingObjects[0] = sparrow;
                flyingObjects[1] = plane;

                for(int i = 0; i < 2; i++){
                        flyingObjects[i].fly();
                }

        }
}
```

Exemple : on peut regrouper les objets qui volent dans un tableau d'objets volants (2/2)

Sorties console

```
Sparrow flying...
Plane flying....
```

- On parle ici de polymorphisme. Le même appel de la méthode *fly()* déclenche des méthodes redéfinies différentes (portant le même nom) de différentes classes dérivées.

Section 4

Les classes abstraites en Java

Classe abstraite

- On peut définir la classe Animal comme étant une classe abstraite avec :
  - des méthodes concrètes communes à toutes les classes qui étendent cette classe.
  - des méthodes abstraites qui nécessiteront des implémentations spécifiques dans chaque sous-classe.
- Une classe abstraite ne peut pas être instanciée.
- L'avantage d'une classe abstraite est principalement de factoriser le code et là encore de définir un super-type, ici Animal.
- On pourra utiliser des fonctionnalités communes aux animaux sans forcément connaître leur sous-type.

Classe abstraite *Animal*

```
public abstract class Animal {

        int age;
        String gender;
        double weight;

        public Animal(int age, String gender, double weight){
                this.age = age;
                this.gender = gender;
                this.weight = weight;
        }

        public void eat(){
                System.out.println("Eating...");
        }

        // Methode abstraite
        public abstract void move();
}
```

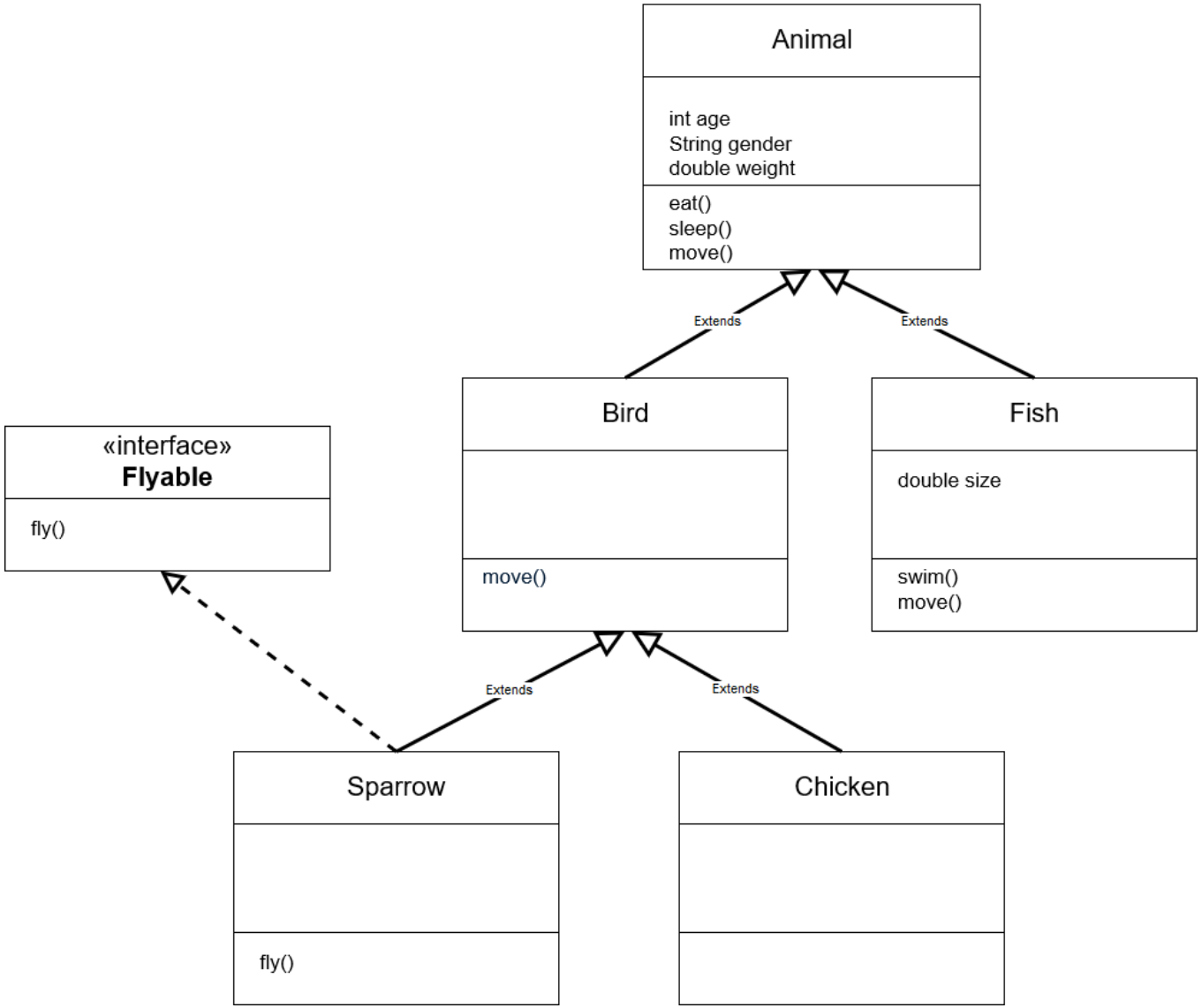
Classe concrète *Fish* implémentant la classe abstraite *Animal*

```
public class Fish extends Animal{

        double size;

        public Fish(int age, String gender, double weight, double size) {
                super(age, gender, weight);
                this.size = size;
        }

        public void swim(){
                System.out.println("Swimming...");
        }

        // Obligation d implementer move() car Fish extends Animal
        public void move(){
                System.out.println("The fish is swimming...");
        }

}
```

Classe concrète *Bird* implémentant la classe abstraite *Animal*

```
public class Bird extends Animal{

        public Bird(int age, String gender, double weight) {
                super(age, gender, weight);
        }

        // Obligation d implementer move() car Bird extends Animal
        public void move() {
                System.out.println("Flapping wings...");

        }

}
```

Diagramme de classes UML simplifié



Test

```
public class Zoo {

        public static void main(String args[]){

                Sparrow Sparrow = new Sparrow(13, "M", 500);
                Sparrow.move();

                Fish fish = new Fish(3, "F", 400, 800);
                fish.move();

        }
}
```

Console

Flapping wings...
The fish is swimming...

Sparrow et *Fish* sont de type parent *Animal*

```
public class Zoo {

        public static void main(String args[]){

                Sparrow sparrow = new Sparrow(13, "M", 500);

                Fish fish = new Fish(3, "F", 400, 800);

                moveAnimals(Sparrow);
                moveAnimals(fish);
        }

        public static void moveAnimals(Animal animal){
                animal.move();
        }
}
```

Sorties console

Flapping wings...
The fish is swimming...

Sparrow et *Fish* sont de type parent *Animal*

- On utilise ici le polymorphisme pour écrire une méthode *moveAnimals(Animal animal)* générique pour tous les animaux.
- une mauvaise solution serait d'utiliser la surchage pour définir une méthode par type d'animal. → Très dur à maintenir si on doit ajouter de nouveaux animaux par la suite...

```
public class Zoo {
    ...
        public static void moveAnimals(Fish fish){
                fish.move();
        }

        public static void moveAnimals(Bird bird){
                bird.move();
        }
        ....
}
```

Classe abstraite ou interface ?

- Point commun : les deux définissent des méthodes abstraites et ne peuvent pas être instanciées
- Différence : des méthodes concrètes peuvent être implémentées dans les classes abstraites, ce qui n'est pas possible dans les interfaces

Classe abstraite ou interface ?

- Pourquoi ne pas utiliser des classes abstraites (dans lesquelles aucune méthode ne serait implémentée) à la place des interfaces ?
  $\rightarrow$ En Java, il n'est possible pour une classe d'hériter que d'une seule classe parent (abstraite ou non), mais d'implémenter plusieurs interfaces
- Quand utiliser une classe abstraite ?
  $\rightarrow$ Les classes abstraites servent à factoriser du code.
- Quand utiliser une interface ?
  $\rightarrow$ Les interfaces servent à définir des *contrats de service*.

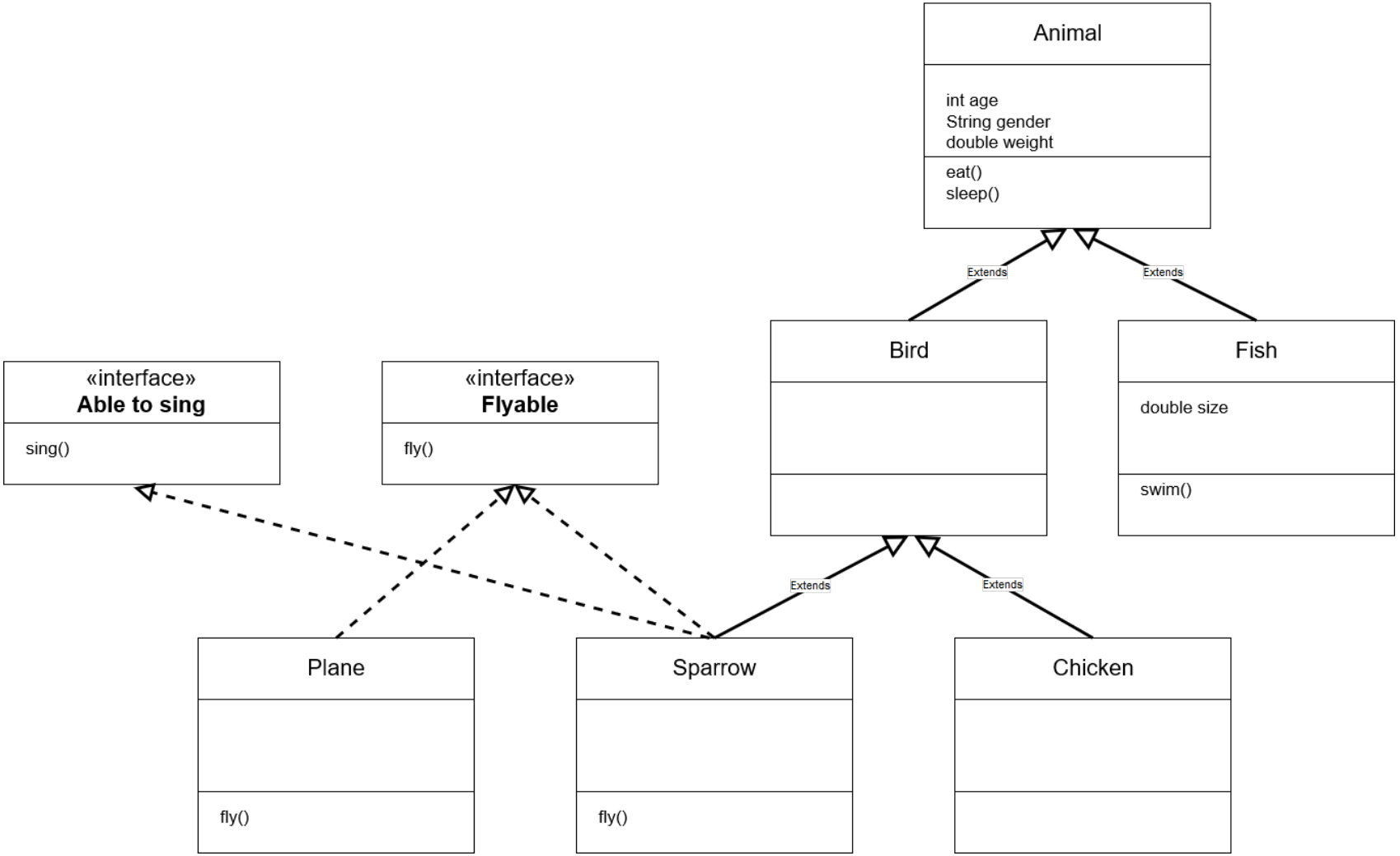



Diagramme de classes avec plusieurs interfaces